Group A: Arrays and Searching Sorting Algorithms

Experiment 2

Title: Employee Salary Sorting and Top Five Highest Salary Identification

Objectives:

- To implement Selection Sort and Bubble Sort algorithms in Python.
- To sort a list of employee salaries in ascending order using both algorithms.
- To identify and display the top five highest salaries from the sorted list.

Problem Statement:

Given a list of employee salaries as floating-point numbers, sort them in ascending order using Selection Sort and Bubble Sort algorithms, and then extract and display the top five highest salaries.

Outcomes:

- A Python program demonstrating the implementation of Selection Sort and Bubble Sort.
- Sorted lists of employee salaries using both algorithms.
- Identification and display of the top five highest salaries.

Software and Hardware Requirements:

- **Software:** Python 3.x interpreter.
- **Hardware:** A computer capable of running Python programs.

Date of Completion: October 6, 2025

Assessment grade/marks and assessor's sign: (To be filled by assessor)

Theory Concept in brief:

- **Selection Sort:**

  This algorithm repeatedly finds the minimum element from the unsorted part of the list and puts it at the beginning. It maintains two subarrays in a given array: one already sorted and one remaining unsorted.

- **Bubble Sort:**

  This algorithm repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until no swaps are needed, indicating that the list is sorted.

Algorithm:

1. Selection Sort:

- Initialize n as the length of the salary list.
- Iterate from i = 0 to n-1:
  a. Set min_idx = i.
  b. Iterate from j = i+1 to n:
  i. If salaries[j] < salaries[min_idx], set min_idx = j.
  c. Swap salaries[i] and salaries[min_idx].

2. Bubble Sort:

- Initialize n as the length of the salary list.
- Iterate from i = 0 to n-1:
  a. Set swapped = False.
  b. Iterate from j = 0 to n-i-1:
  i. If salaries[j] >salaries[j+1], swap salaries[j] and salaries[j+1].
  ii. Set swapped = True.
  c. If swapped is False, break the outer loop (list is sorted).

3. Display Top Five:

- Reverse the sorted list to get descending order.
- Print the first five elements of the reversed list.
  Flowchart: (Cannot be directly represented in text, but would depict the sequential steps of each sorting algorithm and the final display of top salaries.)

Test Cases:

Test Case 1:

- **Input:** salaries = [1500.0, 2500.5, 1200.75, 3000.0, 1800.25, 2800.0, 3500.0, 2000.0]
- **Expected Output (Sorted):** [1200.75, 1500.0, 1800.25, 2000.0, 2500.5, 2800.0, 3000.0, 3500.0]
- **Expected Output (Top 5):** [3500.0, 3000.0, 2800.0, 2500.5, 2000.0]
  Test Data Set: (See Test Case 1)

  Mathematical Model: (Not applicable for this problem, as it focuses on algorithmic implementation rather than mathematical modeling.)

Conclusion/Analysis:

This program successfully demonstrates the implementation of Selection Sort and Bubble Sort algorithms for sorting employee salaries. Both algorithms effectively sort the list in ascending order. After sorting, extracting the top five highest salaries is achieved by reversing the sorted list and accessing the first five elements. The choice between Selection Sort and Bubble Sort depends on factors like data size and performance requirements, with more efficient algorithms typically preferred for larger datasets.

Program:-

```
defselection_sort(arr):
n = len(arr)
foriin range(n):
min_idx = i
forjin range(i + 1, n):
ifarr[j] <arr[min_idx]:
min_idx = j
arr[i], arr[min_idx] = arr[min_idx], arr[i]
returnarr

defbubble_sort(arr):
n = len(arr)
foriin range(n - 1):
swapped = False
forjin range(0, n - i - 1):
ifarr[j] >arr[j + 1]:
arr[j], arr[j + 1] = arr[j + 1], arr[j]
swapped = True
ifnot swapped:
break
returnarr

# Employee salaries
employee_salaries = [1500.0, 2500.5, 1200.75, 3000.0, 1800.25, 2800.0, 3500.0,
2000.0]

# Sort using Selection Sort
salaries_selection_sorted = selection_sort(list(employee_salaries))
print("Salaries sorted with Selection Sort:", salaries_selection_sorted)

# Sort using Bubble Sort
salaries_bubble_sorted = bubble_sort(list(employee_salaries))
print("Salaries sorted with Bubble Sort:", salaries_bubble_sorted)

# Display top five highest salaries (from either sorted list, they will be
the same)
top_five_salaries = sorted(employee_salaries, reverse=True)[:5]
print("Top five highest salaries:", top_five_salaries)
```

Group B Stacks Queues and Linked List

Experiment 3

Title: Real-time Undo/Redo System for Text Editor using Stack Data Structure

Objectives:

- To design and implement an undo/redo mechanism for a text editing application.
- To utilize the Stack data structure effectively to manage document changes for undo and redo operations.
- To ensure real-time responsiveness for undo, redo, and display document state operations.
- To provide a clear and intuitive user experience for managing document revisions.

Problem Statement:

Text editing applications require a robust and efficient undo/redo functionality to allow users to revert or reapply changes. The challenge lies in designing a system that can track document modifications, store them for potential reversal, and reapply them accurately, all while maintaining a responsive user interface. This system must handle multiple changes and their corresponding undo/redo operations in a sequential manner.

Outcomes:

Upon successful implementation, the system will:

- Provide a functional undo operation that reverts the most recent change.
- Provide a functional redo operation that reapplies the most recently undone change.
- Accurately display the current state of the document after any undo or redo action.
- Demonstrate the effective application of the Stack data structure in managing sequential operations.

Software and Hardware Requirements:

- **Software:**
- Operating System: Windows 10/11, macOS, or Linux distribution.
- Programming Language: Python (or Java, C++, etc., depending on implementation choice).
- IDE/Text Editor: VS Code, PyCharm, IntelliJ IDEA, or similar.

Hardware:

- Processor: Intel Core i3 or equivalent.

- RAM: 4GB or more.
- Storage: 100MB free disk space.
  Date of Completion: October 6, 2025

  Assessment grade/marks and assessor's sign: (To be filled by assessor)

Theory Concept in brief:

The core of this system relies on the Stack data structure, a Last-In, First-Out (LIFO) collection. Two stacks are primarily used: an "undo stack" and a "redo stack." When a change is made, it is pushed onto the undo stack. When an undo operation occurs, the change is popped from the undo stack and pushed onto the redo stack. Conversely, a redo operation pops a change from the redo stack and pushes it back onto the undo stack. This mechanism ensures that changes are tracked and reversible in the correct order.

Algorithm:

- **Initialize:** Create two empty

  stacks: undo_stack and redo_stack. Initialize current_document_state.
- Make a Change (e.g., apply_change(new_state)):
  a. Push current_document_state onto undo_stack.
  b. Update current_document_state to new_state.
  c. Clear redo_stack. Undo Action (undo()).

  a. If undo_stack is not empty:
  i. Push current_document_state onto redo_stack.
  ii. Pop previous_state from undo_stack.
  iii. Update current_document_state to previous_state.
  b. Else: Display "No more changes to undo." Redo Action (redo()).

  a. If redo_stack is not empty:
  i. Push current_document_state onto undo_stack.
  ii. Pop next_state from redo_stack.
  iii. Update current_document_state to next_state.
  b. Else: Display "No more changes to redo." Display Document State (display_document()).

  a. Print current_document_state.

Flowchart:

Program:-

```
classTextEditor:
def__init__(self):
self.current_document_state = ""
self.undo_stack = []
self.redo_stack = []

defmake_change(self, new_content):
self.undo_stack.append(self.current_document_state)
self.current_document_state = new_content
self.redo_stack = []  # Clear redo stack on new change
self.display_document_state()

defundo_action(self):
ifself.undo_stack:
self.redo_stack.append(self.current_document_state)
self.current_document_state = self.undo_stack.pop()
self.display_document_state()
else:
            print("Nothing to undo.")

defredo_action(self):
ifself.redo_stack:
self.undo_stack.append(self.current_document_state)
self.current_document_state = self.redo_stack.pop()
self.display_document_state()
else:
            print("Nothing to redo.")

defdisplay_document_state(self):
        print(f"Current Document: '{self.current_document_state}'")

# Usage Example:
editor = TextEditor()
editor.make_change("Hello")
editor.make_change("Hello World")
editor.undo_action()
editor.make_change("Hello Python")
editor.undo_action()
editor.redo_action()
```

Experiment 4

Title: Real-time Event Processing System using Queue Data Structure

Objectives:

- To design and implement a real-time event processing system.
- To utilize a Queue data structure for managing incoming events.
- To enable adding new events to the queue.
- To facilitate processing and removal of the longest-waiting event.
- To provide functionality for displaying all pending events.
- To allow cancellation of events before processing.
  Problem Statement:
  Efficient and timely handling of events in a real-time system is crucial. The challenge lies in managing the sequence of events, ensuring oldest events are processed first, providing visibility into pending events, and allowing for event cancellation, all while maintaining system responsiveness.

Outcomes:

- A functional real-time event processing system capable of handling event queues.
- Implementation of add, process, display, and cancel event operations.
- Demonstrable use of a Queue data structure for First-In, First-Out (FIFO) event management.

Software and Hardware Requirements:

- ## Software:
- Operating System: Windows 10/11, macOS, or Linux distribution.
- Programming Language: Python 3.x (or Java, C++, etc., depending on chosen implementation).
- Development Environment: VS Code, PyCharm, IntelliJ IDEA, or similar IDE.

Hardware:

- Processor: Intel Core i3/AMD Ryzen 3 or higher.
- RAM: 4GB or more.
- Storage: Minimum 100MB free space for project files.
  Date of Completion: [Insert Date of Completion]

Theory Concept in brief:

A Queue is a linear data structure that follows the First-In, First-Out (FIFO) principle. Events are added to the rear (enqueue) and removed from the front (dequeue). This structure ensures that events are processed in the order they are received, which is essential for real-time systems where chronological processing is often required.

Algorithm:

- **Initialize Queue:** Create an empty queue to store events.
- **Add Event:**
  - Take event details as input.
  - Enqueue the event into the queue.
- **Process Next Event:**
  - If the queue is not empty, dequeue the event from the front.
  - Perform processing logic for the dequeued event.
- **Display Pending Events:**
  - Iterate through the queue from front to rear.
  - Print details of each event in the queue.
- **Cancel an Event:**
  - Take the event identifier to be canceled as input.
  - Search for the event in the queue.
  - If found and not yet processed (i.e., not at the front if processing is immediate upon dequeue), remove the event from its current position in the queue. This might require re-constructing the queue or using a data structure that supports efficient deletion from the middle.

Flowchart:

[A detailed flowchart depicting the main functionalities (Add, Process, Display, Cancel) and their decision points and process flows would be included here.]

Test Cases:

- **Add Multiple Events:** Add events A, B, C, D to the queue.
- **Process Events:** Process the next two events (A, B).
- **Display Pending Events:** Verify that C and D are displayed.
- **Cancel Event:** Cancel event C.
- **Display Pending Events (after cancellation):** Verify that only D is displayed.

- **Process Remaining Event:** Process the next event (D).
- **Display Pending Events (empty queue):** Verify that no events are displayed.
- **Attempt to Process Empty Queue:** Verify graceful handling.
- **Attempt to Cancel Non-Existent Event:** Verify graceful handling.

Test Data Set (if applicable):

- Event IDs: E001, E002, E003, E004
- Event Descriptions: "User Login", "Database Update", "File Upload", "API Request"
- Event Timestamps: (simulated or actual timestamps)

Mathematical Model (if applicable):

The system implicitly models a queuing system where events arrive and are served. Performance metrics such as average waiting time, queue length, and throughput could be analyzed using queuing theory, but for this basic implementation, a formal mathematical model is not strictly required.

Conclusion/Analysis of Implementation:

The implementation successfully demonstrates a real-time event processing system using a Queue data structure. The FIFO nature of the queue ensures proper event ordering, and the provided functionalities address the core requirements of adding, processing, displaying, and canceling events. The system provides a foundational framework for more complex real-time event management scenarios. Further enhancements could include priority queues, event logging, and distributed event handling.

Group C – Hashing

Experiment 5

Title: Hash Table Implementation with Chaining for Collision Resolution

Objectives:

- To understand the concept of hash tables and their applications.
- To implement a hash table using the division method as a hash function.
- To implement chaining as a collision resolution technique.
- To implement essential hash table operations: Insert, Search, and Delete.

Problem Statement:

Develop a hash table data structure of size 10 that utilizes the division method for hashing keys. In the event of a collision, implement chaining to store multiple elements at the same index. The implementation must support insertion of key-value pairs, searching for values based on keys, and deletion of key-value pairs.

Outcomes:

- A functional hash table implementation capable of storing, retrieving, and deleting data efficiently.
- A clear understanding of hashing, collision resolution (chaining), and fundamental hash table operations.
- Proficiency in implementing data structures in a chosen programming language.

Software and Hardware Requirements:

- **Software:**
  - Operating System: Windows 10/11, macOS, or Linux distribution.
  - Programming Language: Python 3.x, Java Development Kit (JDK), C++ compiler (e.g., GCC), etc.
  - Integrated Development Environment (IDE) or Text Editor: VS Code, IntelliJ IDEA, Eclipse, PyCharm, etc.

Hardware:

- Processor: Intel Core i3 or equivalent.
- RAM: 4GB or more.
- Storage: Minimum 100MB free space.

Theory Concept in Brief:

A hash table is a data structure that maps keys to values using a hash function to compute an index, also known as a hash code, into an array of buckets or slots. The division method calculates the hash index as `key % table_size`. Chaining is a collision resolution technique where each bucket in the hash table points to a linked list (or similar data structure) containing

all key-value pairs that hash to that same index. This allows multiple elements to be stored at the same hash index without overwriting existing data.
Algorithm:

- **Hash Table Structure:**

  An array of size 10, where each element is a reference to a linked list (or list in Python).

- **Hash Function (Division Method):**

  `hash_index = key % 10`

- **Insert(key, value):**
  - Calculate `hash_index = key % 10`.
  - If the linked list at `hash_index` is empty, create a new linked list and add the `(key, value)` pair.
  - Otherwise, traverse the linked list at `hash_index`. If `key` already exists, update its `value`. Else, add the `(key, value)` pair to the end of the linked list.

Search(key):
- Calculate `hash_index = key % 10`.
- Traverse the linked list at `hash_index`.
- If `key` is found, return its associated `value`.
- If `key` is not found after traversing the entire list, return an indication of not found (e.g., `None`).

Delete(key):
- Calculate `hash_index = key % 10`.
- Traverse the linked list at `hash_index`.
- If `key` is found, remove the `(key, value)` pair from the linked list.
- If `key` is not found, indicate that the key was not present.

Flowchart:

Test Cases:

- **Insert:**
  - Insert (5, "Apple"), (15, "Banana"), (25, "Cherry") - all hash to index 5.
  - Insert (3, "Dog"), (13, "Elephant") - all hash to index 3.

Search:

- Search for "Apple" (key 5). Expected: "Apple".
- Search for "Cherry" (key 25). Expected: "Cherry".
- Search for "Cat" (key 7). Expected: Not found.

Delete:

- Delete "Banana" (key 15). Expected: "Banana" removed from index 5.
- Delete "Dog" (key 3). Expected: "Dog" removed from index 3.
- Delete "Zebra" (key 99). Expected: Key not found.

Test Data Set (if applicable):

Code

```
Key, Value
5, Apple
15, Banana
25, Cherry
3, Dog
13, Elephant
7, Grape
17, Fig
```

Mathematical Model (if applicable):

- **Hash Function:**
  h(k)=k(modm),
  where is the key
  $m$is the table size (10 in this case).
- **Load Factor:**

  α=n/m,where
  $n$is the number of elements and mm
  $m$is the table size. In chaining,
  αcan be greater than 1.

Conclusion/Analysis of Implementation:

The implementation successfully demonstrates a hash table using the division method and chaining for collision resolution. The Insert, Search, and Delete operations are functional, providing efficient data storage and retrieval. Chaining effectively handles collisions, preventing data loss and maintaining the integrity of the hash table. Performance for these operations is generally O(1)cap O open paren 1 close paren$O(1)$ on average, but can degrade to O(n)cap O open paren n close paren$O(n)$in the worst-case scenario where all elements hash to the same bucket, resulting in a long linked list traversal. Optimizing the hash function and table size can mitigate this performance degradation.

Experiment 6

Title: Implementation of a Fixed-Size Hash Table with Division Method and Linear Probing
Objectives:

- To design and implement a hash table data structure of a fixed size.
- To utilize the division method for calculating hash indices.
- To implement linear probing for resolving collisions during key insertion and search.
- To enable users to perform essential hash table operations: insert, search, delete, and display.

Problem Statement:
Efficient storage and retrieval of data are crucial in many applications. This project addresses the need for a data structure that allows for fast average-case performance for insertion, deletion, and search operations, even with a fixed memory allocation. Collisions, a common issue in hashing, need to be effectively managed to maintain performance.
Outcomes:
A functional hash table implementation capable of storing integer keys, performing insertions, searches, deletions, and displaying its current state. The implementation will demonstrate the use of division method hashing and linear probing for collision resolution.
Software and Hardware Requirements:

- **Software:**

  C/C++ compiler (e.g., GCC), Text editor or Integrated Development Environment (IDE) (e.g., VS Code, Code::Blocks).

- **Hardware:**

  Standard personal computer with sufficient RAM and storage for development and execution.

Theory Concept in brief:

A hash table is a data structure that maps keys to values using a hash function. The division method for hashing calculates the hash index as `key % table_size`. Collision resolution is necessary when two different keys map to the same index. Linear probing is a collision resolution technique where, upon a collision, the algorithm sequentially searches for the next available empty slot in the table. Deletion typically involves marking a slot as "deleted" to maintain the integrity of subsequent searches.

Algorithm:

Code:-

```
// Hash Function (Division Method)
hash(key, table_size):
returnkey % table_size

// Insert Key
insert(key, hash_table, table_size):
index = hash(key, table_size)
whilehash_table[index] isnot empty andhash_table[index] isnot DELETED:
index = (index + 1) % table_size
hash_table[index] = key

// Search Key
search(key, hash_table, table_size):
index = hash(key, table_size)
start_index = index
whilehash_table[index] isnot empty:
ifhash_table[index] == key:
returnindex// Key found
index = (index + 1) % table_size
ifindex == start_index: // Full circle without finding
break
return -1// Key not found

// Delete Key
delete(key, hash_table, table_size):
index = search(key, hash_table, table_size)
ifindex != -1:
hash_table[index] = DELETED // Mark as deleted

// Display Table
display(hash_table, table_size):
forifrom0to table_size-1:
        print hash_table[i]
```

Flowchart:

Test Cases:

- **Insert:**
    - Insert keys: 10, 20, 30 (assuming `table_size = 5`).
    - Expected: Keys placed at calculated indices or subsequent available slots.

Search:

- Search for an existing key (e.g., 20).
- Search for a non-existing key (e.g., 15).

Delete:

- Delete an existing key (e.g., 10).
- Attempt to delete a non-existing key (e.g., 25).
- Verify subsequent searches for the deleted key.

Collision Handling:

- Insert keys that cause collisions (e.g., 5, 10, 15 with `table_size = 5`).
- Verify linear probing correctly places subsequent keys.

Display:

- Display the table after various insert/delete operations.

Test Data Set (if applicable):
For a `table_size = 7`:

- Insert: 5, 12, 19, 26, 3, 10, 17
- Delete: 12, 10
- Search: 19, 3, 25

Mathematical Model (if applicable):
The hash function is defined as

h(k)=k(modm), where
*k*is the key and
*m*is the table size.
The probe sequence for linear probing is h(k),(h(k)+1)(modm),(h(k)+2)(modm),…h of k comma open paren h of k plus 1 close paren space open paren mod m close paren comma open paren h of k plus 2 close paren space open paren mod m close paren comma …
$h(k),(h(k)+1)(\mathrm{mod}m),(h(k)+2)(\mathrm{mod}m),\ldots$
.

Conclusion/Analysis of Design and Implementation:

The design successfully implements a fixed-size hash table using the division method for hashing and linear probing for collision resolution. The system allows for basic hash table operations. While linear probing is simple to implement, it can suffer from primary clustering, potentially degrading performance for large numbers of collisions. Future improvements could explore alternative collision resolution strategies like quadratic probing or separate chaining to mitigate this issue. The fixed size also limits scalability, suggesting dynamic resizing as a potential enhancement for more robust applications.

Group D: Graphs and Trees

Experiment 7

Title: Implementation of Binary Search Tree Operations

Objectives:

To understand the structure and properties of a Binary Search Tree (BST).
To implement fundamental operations on a BST: insertion, deletion, display (in-order, pre-order, post-order traversal), and search.
To analyze the time and space complexity of these operations.

Problem Statement:

Develop a program that effectively manages a collection of data using a Binary Search Tree, allowing users to perform various operations such as adding new elements, removing existing elements, searching for specific elements, and viewing the tree's contents in different traversal orders.

Outcomes:

A functional program capable of performing all specified BST operations.
A clear understanding of BST data structure and its applications.
Proficiency in implementing recursive algorithms for tree traversals.

Software and Hardware Requirements:

Software: C++ compiler (e.g., g++), Integrated Development Environment (IDE) like VS Code or Code::Blocks.
Hardware: A computer system with a minimum of 4GB RAM and a modern processor.

Date of Completion: October 6, 2025

Assessment Grade/Marks and Assessor's Sign: (To be filled by assessor)

Theory Concept in Brief:

A Binary Search Tree (BST) is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- Both the left and right subtrees must also be binary search trees.
- There must be no duplicate nodes.

Algorithm:

Insertion:

- If the tree is empty, create a new node and make it the root.
- If the new node's value is less than the current node's value, recursively insert into the left subtree.
- If the new node's value is greater than the current node's value, recursively insert into the right subtree.

Deletion:

- Find the node to be deleted.
- **Case 1: Node has no children:** Simply remove the node.
- **Case 2: Node has one child:** Replace the node with its child.
- **Case 3: Node has two children:** Find the in-order successor (smallest in right subtree) or predecessor (largest in left subtree), copy its value to the node to be deleted, and then delete the successor/predecessor.
  Display (In-order Traversal): Traverse the left subtree, Visit the root, and Traverse the right subtree.

Search:

- If the tree is empty or the current node's value matches the search value, return the current node.
- If the search value is less than the current node's value, recursively search the left subtree.
- If the search value is greater than the current node's value, recursively search the right subtree.
  Flowchart: (A visual representation of the algorithms for insertion, deletion, display, and search would be presented here, using standard flowchart symbols for processes, decisions, input/output, and start/end points.)

Test Cases:

- **Insertion:** Insert elements.
- **Search:** Search for existing element (40), non-existing element (90).
- **Deletion:** Delete a leaf node (20), a node with one child (60), a node with two children (50).
- **Display:** Perform in-order, pre-order, and post-order traversals on the tree after insertions and deletions.

Test Data Set:

Initial tree: empty
Insertion sequence: 50, 30, 70, 20, 40, 60, 80
Search values: 40, 90
Deletion values: 20, 60, 50

Mathematical Model (if applicable):

The height of a BST can impact the time complexity of operations. In the worst case (skewed tree), the height can be O(N), leading to O(N) complexity for operations. In the best case (balanced tree), the height is O(log N), resulting in O(log N) complexity.

Conclusion/Analysis of Implementation:

The implementation successfully demonstrates the core operations of a Binary Search Tree. The choice of recursive algorithms for insertion, deletion, and traversals provides a concise and elegant solution. The time complexity of operations is dependent on the tree's balance, ranging from O(log N) for a balanced tree to O(N) for a skewed tree. Further enhancements could include self-balancing mechanisms like AVL trees or Red-Black trees to guarantee O(log N) performance for all operations. The test cases confirm the correctness of each implemented operation.

Experiment 8

Title: Expression Tree Construction, Non-Recursive Post-Order Traversal, and Deletion

Objectives:

- To construct an expression tree from a given prefix expression.
- To implement a non-recursive post-order traversal algorithm for the constructed expression tree.
- To implement a function for the complete deletion of the expression tree.
  Problem Statement:
  Given a prefix expression (e.g., `+--a*bc/def`), construct its corresponding expression tree. Subsequently, perform a non-recursive post-order traversal of this tree, and finally, delete the entire tree to free up allocated memory.

Outcomes:

- A correctly constructed expression tree representing the given prefix expression.
- Successful non-recursive post-order traversal, printing the expression in postfix form.
- Complete and efficient deletion of the expression tree, preventing memory leaks.
  Software and Hardware Requirements:
  Software: C++ compiler (e.g., GCC), Integrated Development Environment (IDE) like VS Code or Code::Blocks.
  Hardware: Standard personal computer with sufficient RAM and processing power for compilation and execution.

  Date of Completion: [Insert Date]

  Assessment grade/marks and assessor's sign: [Grade/Marks] / [Assessor's Signature]

  Theory Concept in brief:
  An expression tree is a binary tree used to represent arithmetic or logical expressions. Internal nodes are operators, and leaf nodes are operands. Prefix expressions (Polish notation) place operators before their operands. Post-order traversal visits the left subtree, then the right subtree, then the root. Non-recursive traversal typically utilizes a stack. Tree deletion involves traversing the tree and freeing memory for each node.

  Algorithm: Construct Expression Tree.

  a. Read the prefix expression from right to left.
  b. Use a stack to store nodes.
  c. If a character is an operand, create a new node and push it onto the stack.
  d. If a character is an operator, create a new node. Pop two nodes from the stack (right

child then left child) and assign them as children of the operator node. Push the operator node onto the stack.
e. The final node on the stack is the root of the expression tree. Non-Recursive Post-Order Traversal.

a. Use two stacks: `s1` and `s2`.
b. Push the root onto `s1`.
c. While `s1` is not empty:
i. Pop a node from `s1` and push it onto `s2`.
ii. If the popped node has a left child, push it onto `s1`.
iii. If the popped node has a right child, push it onto `s1`.
d. While `s2` is not empty, pop and print the node's data. Delete Tree.

a. Implement a recursive function `deleteTree(node)`:
i. If `node` is NULL, return.
ii. Recursively call `deleteTree(node->left)`.
iii. Recursively call `deleteTree(node->right)`.
iv. Delete `node`.

Flowchart:

Test Cases:

Prefix Expression	Expected Post-order Traversal
`+--a*bc/def`	`abc*de/f--+`
`*+ab-cd`	`ab+cd-*`
`Abc`	`abc`

Test Data Set (if applicable):
Not applicable for this problem, as the input is a single prefix expression.

Mathematical Model (if applicable):
Not applicable. The problem focuses on data structures and algorithms.

Conclusion/Analysis:

The construction of the expression tree from a prefix expression demonstrates the utility of stacks in parsing and tree building. The non-recursive post-order traversal highlights an alternative to recursive approaches, also leveraging stack data structures. The tree deletion ensures proper memory management, preventing resource leaks. The chosen algorithms effectively address the problem, providing a clear and functional solution for expression tree manipulation.